

1. What is Python?

Explain what Python is and describe its major features.

Points to consider:

- High-level programming language
 - Interpreted language
 - Dynamically typed language
 - Object-oriented programming support
 - Large standard library
 - Cross-platform support
 - Large ecosystem of third-party libraries
-

2. What are the main features of Python?

Explain at least five important features of Python.

For example:

- Simple syntax
 - Readability
 - Dynamic typing
 - Automatic memory management
 - Object-oriented programming
 - Extensive libraries
 - Platform independence
-

3. What is the difference between interpreted and compiled languages?

Explain how Python code is executed.

Discuss:

- Source code
 - Bytecode
 - Python Virtual Machine
 - Interpretation/execution
-

4. What are variables in Python?

Explain how variables are created in Python.

Example:

```
name = "Amit"  
age = 25
```

```
salary = 50000
```

Also explain why Python does not require explicit variable type declarations.

5. What are the built-in data types in Python?

Explain the following data types with examples:

- `int`
 - `float`
 - `complex`
 - `bool`
 - `str`
 - `list`
 - `tuple`
 - `set`
 - `dict`
 - `NoneType`
-

6. What is the difference between mutable and immutable data types?

Explain the difference between mutable and immutable objects.

Give examples of:

- Mutable objects
- Immutable objects

Also explain why lists are mutable while tuples are immutable.

7. What is type casting in Python?

Explain how to convert one data type into another.

Demonstrate:

```
int()  
float()  
str()  
list()  
tuple()  
set()
```

Example:

```
number = "100"  
result = int(number)
```

8. What is the difference between `==` and `is`?

Explain the difference between:

```
a == b
```

and:

```
a is b
```

Give examples demonstrating value comparison and object identity.

9. What is `None` in Python?

Explain the meaning of `None`.

What is the difference between:

```
x = None
```

and:

```
x = 0
```

10. What are Python operators?

Explain the major categories of operators in Python.

Include examples of:

- Arithmetic operators
- Comparison operators
- Logical operators
- Assignment operators
- Membership operators
- Identity operators

- Bitwise operators
-

11. What is the difference between `if`, `elif`, and `else`?

Write a Python program that accepts a student's marks and prints:

- `A` for marks ≥ 90
 - `B` for marks ≥ 75
 - `C` for marks ≥ 60
 - `D` for marks ≥ 40
 - `Fail` otherwise
-

12. What is the difference between `for` and `while` loops?

Explain when you would use each loop.

Write an example of both.

13. What is the purpose of `break`, `continue`, and `pass`?

Explain the difference between:

```
break
continue
pass
```

Provide a simple example for each.

14. Write a program to find the factorial of a number.

Create a Python program that accepts an integer and calculates its factorial using a loop.

Example:

```
Input: 5
Output: 120
```

15. Write a program to check whether a number is prime.

Accept a number from the user and determine whether it is prime.

Example:

Input: 17
Output: Prime

16. What is a function in Python?

Explain:

- Why functions are used
- Function definition
- Function call
- Parameters
- Return values

Example:

```
def add(a, b):  
    return a + b
```

17. What is the difference between parameters and arguments?

Explain the difference using:

```
def greet(name):  
    print("Hello", name)  
  
greet("Amit")
```

Identify the parameter and argument.

18. What are default arguments in Python?

Explain how default arguments work.

Example:

```
def greet(name="Guest"):  
    print("Hello", name)
```

What happens when the function is called with and without an argument?

19. What are `*args` and `**kwargs`?

Explain the purpose of:

```
*args
```

and:

```
**kwargs
```

Write a function that accepts any number of positional and keyword arguments.

20. What is a lambda function?

Explain lambda functions and write examples for:

- Addition
- Squaring a number
- Checking whether a number is even

Example:

```
square = lambda x: x * x
```

21. What is a list in Python?

Explain the characteristics of lists.

Write a program that:

- Creates a list
 - Adds an element
 - Removes an element
 - Updates an element
 - Prints the list
-

22. What is the difference between a list and a tuple?

Compare lists and tuples based on:

- Syntax
- Mutability
- Performance

- Use cases
- Memory usage

Give examples.

23. What is a tuple?

Create a tuple containing five programming languages.

Write programs to:

- Access elements
 - Find its length
 - Check whether an item exists
 - Count an item
-

24. What is a set?

Explain the characteristics of sets.

Create two sets and demonstrate:

- Union
 - Intersection
 - Difference
 - Symmetric difference
-

25. What is a dictionary?

Explain dictionaries and key-value pairs.

Create a dictionary representing a student:

```
student = {  
    "name": "Amit",  
    "age": 25,  
    "marks": 85  
}
```

Write code to:

- Access values
 - Add a key
 - Update a value
 - Delete a key
-

26. How do you remove duplicates from a list?

Given:

```
numbers = [1, 2, 2, 3, 4, 4, 5, 5]
```

Write a Python program to remove duplicate elements.

Solve it using:

1. A set
 2. A loop
-

27. What is list slicing?

Explain list slicing using:

```
numbers = [10, 20, 30, 40, 50, 60, 70]
```

Write expressions to obtain:

- First three elements
 - Last three elements
 - Every second element
 - Reversed list
-

28. What is list comprehension?

Explain list comprehension.

Convert this loop into a list comprehension:

```
numbers = []  
  
for i in range(1, 11):  
    numbers.append(i * i)
```

29. How do you sort a list?

Explain the difference between:

```
sort()
```

and:

```
sorted()
```

Demonstrate both with examples.

30. How do you count the frequency of elements in a list?

Given:

```
numbers = [1, 2, 2, 3, 3, 3, 4]
```

Create a dictionary containing the frequency of each number.

Expected result:

```
{
    1: 1,
    2: 2,
    3: 3,
    4: 1
}
```

31. What is Object-Oriented Programming?

Explain the concept of OOP.

Describe:

- Class
- Object
- Attribute
- Method

Give a simple Python example.

32. What is a class and what is an object?

Create a `Student` class with:

- Name
- Age

- Marks

Create two student objects and display their information.

33. What is the `__init__()` method?

Explain the purpose of the constructor method:

```
__init__()
```

Create a class called `Employee` and initialize:

- Name
 - Salary
 - Department
-

34. What is `self` in Python?

Explain the purpose of `self`.

Why is `self` used inside instance methods?

Provide an example.

35. What is inheritance?

Explain inheritance in Python.

Create:

```
Animal
|
+-- Dog
```

The `Animal` class should have a method called `speak()`.

The `Dog` class should inherit from `Animal`.

36. What is method overriding?

Create a parent class and child class where both define the same method.

Demonstrate how the child class overrides the parent's method.

37. What is encapsulation?

Explain encapsulation in Python.

Demonstrate the use of:

```
_public  
__private
```

attributes.

38. What is polymorphism?

Explain polymorphism with a simple example.

Demonstrate how the same method name can behave differently for different objects.

39. What is NumPy?

Explain what NumPy is and why it is widely used in Python for numerical computing.

Discuss:

- NumPy arrays
 - Performance
 - Vectorized operations
 - Mathematical operations
 - Multi-dimensional arrays
-

40. What is the difference between a Python list and a NumPy array?

Compare Python lists and NumPy arrays based on:

- Data types
 - Memory
 - Performance
 - Mathematical operations
 - Use cases
-

41. How do you create a NumPy array?

Write code to create:

1. A one-dimensional array
2. A two-dimensional array
3. An array containing zeros

4. An array containing ones
5. An array containing numbers from 1 to 10

Example:

```
import numpy as np

arr = np.array([1, 2, 3, 4, 5])
```

42. What is vectorization in NumPy?

Explain vectorized operations.

Compare:

```
numbers = [1, 2, 3, 4, 5]
```

with a NumPy array.

Demonstrate how to multiply every element by 2 without explicitly writing a loop.

43. What are the **shape**, **size**, and **ndim** properties in NumPy?

Given:

```
import numpy as np

arr = np.array([
    [1, 2, 3],
    [4, 5, 6]
])
```

Find:

- Shape
- Number of dimensions
- Total number of elements

44. What is Pandas?

Explain what Pandas is and why it is used in data analysis.

Discuss:

- Series
 - DataFrame
 - Data cleaning
 - Data filtering
 - Data aggregation
 - CSV/Excel data handling
-

45. What is a Pandas Series?

Create a Pandas Series containing five numbers.

Explain:

- Index
- Values
- Accessing elements

Example:

```
import pandas as pd

s = pd.Series([10, 20, 30, 40, 50])
```

46. What is a Pandas DataFrame?

Create a DataFrame containing:

- Name
- Age
- Marks

for five students.

Print the DataFrame.

47. How do you filter data in Pandas?

Given a DataFrame containing student marks, filter students who scored more than 75.

Example:

```
df[df["marks"] > 75]
```

Explain what this expression does.

48. How do you handle missing values in Pandas?

Explain the purpose of:

```
isnull()
```

```
dropna()
```

```
fillna()
```

Create a DataFrame containing missing values and demonstrate at least two ways to handle them.

49. What is Matplotlib?

Explain what Matplotlib is and why it is used.

Discuss common chart types:

- Line chart
 - Bar chart
 - Scatter plot
 - Histogram
 - Pie chart
-

50. How do you create a basic chart using Matplotlib?

Write a Python program using Matplotlib to create a line chart showing monthly sales.

Use data such as:

```
months = ["Jan", "Feb", "Mar", "Apr", "May"]  
sales = [100, 150, 120, 180, 220]
```

The chart should contain:

- Title
- X-axis label
- Y-axis label
- Grid
- Appropriate markers

Example structure:

```
import matplotlib.pyplot as plt

months = ["Jan", "Feb", "Mar", "Apr", "May"]
sales = [100, 150, 120, 180, 220]

plt.plot(months, sales, marker="o")
plt.title("Monthly Sales")
plt.xlabel("Month")
plt.ylabel("Sales")
plt.grid(True)
plt.show()
```
